

MIGRATION AS KNOWLEDGE COMPILATION

A graph intermediate representation for grey-box, agent-guided legacy modernization with DeepSeek Harness

Working white paper | Version 0.3 | 23 August 2026

Thesis

Treat migration as a compiler pipeline whose source language is the behavior of the legacy system. Grey-box end-to-end exploration samples a high-dimensional scenario/context space; Monte Carlo observations and derivative-like finite differences expose discontinuity surfaces; a Migration Graph IR preserves behavior planes, boundary adjacency, evidence, uncertainty, and effects; DeepSeek Harness interprets that incomplete IR during exploration; and stable graph regions are compiled into target-specific implementations, tests, formal models, workflows, or documentation.

The architecture has three first-class components:

- DeepSeek Harness (DSH) as the execution, plugin, session, and multi-agent substrate.
- Grey-box E2E exploration as the mechanism for acquiring behavioral knowledge from the running legacy system.
- Migration Graph IR (MGIR) as the durable, backend-neutral representation of discovered semantics and migration intent.

Core distinction

The graph is not a transcript, not a control-flow graph, and not a migration checklist. It is an executable quotient of a behavioral hyperspace: nodes represent behavior planes or regions over which one semantic law holds, edges represent reachable crossings between those regions, and boundary evidence records where behavior changes. It also carries what is known, hypothesized, witnessed, and still unexplored.

Executive Summary

Legacy migration is primarily an epistemic problem before it is a code-generation problem. The old system often contains years of undocumented semantics distributed across code branches, configuration, data quirks, external effects, operational conventions, and bug-compatible behavior. A model can rewrite syntax quickly; it cannot safely migrate behavior that has never been discovered.

This paper proposes a knowledge-first architecture. Instead of asking an agent to infer the legacy system from source code alone, DSH runs controlled end-to-end scenarios against the legacy implementation. The exploration is grey-box: externally visible outcomes remain the semantic authority, while internal traces, coverage, logs, data-flow hints, static analysis, and effect instrumentation guide where to probe next and help map behavioral divergences back to implementation regions.

The knowledge acquired from those runs is written into a Migration Graph IR. The IR is deliberately partial and evidence-carrying. Its domain is a behavioral hyperspace formed by scenario parameters, relevant initial state, environment/configuration, and sequence or time features. A node is not merely one point-state: it denotes a behavior plane or region - a connected equivalence class over which the same migration-relevant behavioral law holds. Edges denote reachable transitions or adjacency across boundary surfaces, including guards, effects, observations, code correlations, confidence, and provenance. Unknown regions are represented explicitly as frontiers rather than silently treated as impossible.

Because that hyperspace can be very large, exploration is statistical and adaptive. Monte Carlo sampling supplies broad observations and reveals the empirical mass of behavior regions; nearby observations with different canonical signatures seed candidate boundaries. Local finite differences or random-direction perturbations then estimate a semantic change field. The derivative of a discontinuous behavior is not defined at the boundary; the useful signal is precisely that finite-difference magnitude spikes as a probe straddles a jump. The explorer concentrates subsequent samples around those high-change neighborhoods and refines a local guard or boundary surface.

During discovery, the graph is interpreted. An interpreter selects a frontier or uncertainty, generates the next probe, executes it through DSH, compares behavior signatures, and updates the graph. During migration, sufficiently stable graph subsets are normalized and lowered through a compiler pipeline. Backends can emit conformance tests, target-language code skeletons or domain logic, DSH workflows, formal transition systems for Lean or TLA+, migration plans, diagrams, and other artifacts for which a semantic lowering is defined.

The result is a migration process that continuously turns uncertainty into durable, queryable, replayable knowledge. The legacy application becomes an oracle from which an evidence-backed specification is mined; the graph becomes the transferable specification; and the new system is validated against that specification rather than against an agent's memory of the codebase.

One-sentence architecture

DSH executes and records; grey-box E2E testing discovers; MGIR remembers and reasons; the interpreter explores; the compiler emits.

Document map

Section	Focus
1-2	Why migrations fail as knowledge problems; design principles.
3-5	MGIR semantics, behavioral hyperspace/planes, and evidence/provenance model.
6-8	Grey-box discovery, Monte Carlo observations, derivative boundary search, and state inference.
9-10	DSH integration and interpreter/compiler split.
11-13	Backends, migration workflow, conformance and pure-function extraction.

Section	Focus
14-16	Worked example, implementation roadmap, research position and limits.

1. Migration Is a Knowledge Acquisition Problem

A legacy system is rarely equivalent to its current source tree. Its actual semantics are the intersection of code, persisted data, configuration, external services, operational history, deployment topology, and the set of inputs users have learned to rely on. Migration fails when the rewrite process treats source code as a complete specification.

1.1 Why source-only migration is insufficient

- Important behavior may exist only at integration boundaries: database defaults, serialized formats, retry timing, exception mapping, queue ordering, external API quirks, or environment-specific configuration.
- Branch meaning is often implicit. A condition such as `if (x > 10)` is easy to see statically; why that condition exists, which user scenario activates it, and what downstream effects define compatibility may not be.
- Unit tests are frequently architecture-coupled. They preserve internal decomposition that the migration may intentionally replace.
- Documentation often describes intended behavior, while production users depend on observed behavior - including historically accidental but compatibility-relevant behavior.
- LLM context is transient and lossy. Even a capable model may rediscover the same branch repeatedly or forget an unexplored edge unless knowledge is externalized.

1.2 Why E2E scenarios are the right semantic anchor

End-to-end scenarios survive architectural change. They describe the system in terms of externally meaningful state, input, output, and effect. That makes them a stronger migration oracle than source-level units. The limitation of pure black-box E2E testing is localization: it can tell us that behavior changed, but not efficiently where the relevant internal branch lives. Grey-box instrumentation resolves that tension.

2. Core Design: DSH + Grey-Box Discovery + MGIR

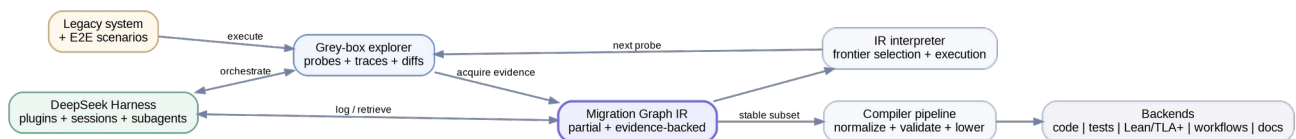


Figure 1. The proposed architecture. DSH is the execution substrate; grey-box testing acquires semantics; MGIR is the durable intermediate representation; interpretation drives exploration and compilation emits backend artifacts.

The design deliberately avoids creating another harness. DeepSeek Harness already provides a plugin-composed execution environment with sessions, tools, sandboxes, subagents, goals, workflows, and a traceable append-only event stream. Its architecture makes the model adapter, tools, session log, and agent loop replaceable capabilities, while Cordis provides the underlying plugin/context composition mechanisms [1][2].

The proposal is therefore an extension layer over DSH: a set of plugins and data contracts that make migration knowledge first-class. DSH supplies the runtime; MGIR supplies the long-lived semantic object around which exploration and compilation operate.

Design rule

Do not encode migration knowledge only in prompts or chat history. Every claim that matters to compatibility should become a graph fact linked to evidence.

3. What the Migration Graph IR Represents

MGIR is a typed, evidence-bearing graph. It represents behavioral regions and transitions discovered from a legacy system, plus migration-oriented annotations that allow those semantics to be interpreted, searched, validated, and lowered to different targets. The graph is a discrete adjacency representation of a much larger behavioral hyperspace.

3.1 The graph is semantic, not syntactic

A static control-flow graph contains every syntactic branch that an analyzer can resolve. MGIR should not. A branch enters MGIR when it corresponds to a behaviorally distinguishable region, transition, boundary, effect, or compatibility obligation. Static code locations are evidence and localization metadata, not the identity of the graph itself.

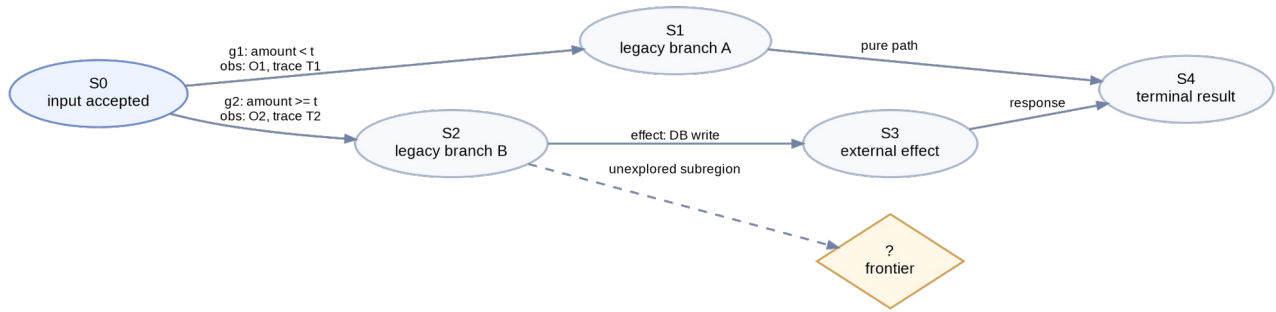


Figure 2. Example semantic state/branch graph. A node captures a witnessed semantic state class; edges carry guards, observations, effects, and provenance. Dashed frontiers make unknown regions explicit.

3.2 A compact formalization

Let the exploratory graph be:

MGIR = (N, E, I, H, P, F)

- N semantic state classes
- E witnessed or hypothesized transitions
- I invariants and compatibility obligations
- H hypotheses: guards, partitions, causal explanations
- P provenance/evidence graph
- F unresolved frontiers and unknown regions

A transition can be represented as:

$t = (\text{src}, \text{guard}, \text{action}, \text{effects}, \text{observation}, \text{dst}, \text{confidence}, \text{evidenceRefs})$

The important departure from a conventional compiler IR is that fields may be incomplete or uncertain during exploration. Unknown is a valid value. Confidence and provenance are part of the semantics, not optional metadata.

3.3 Nodes as behavior planes in a semantic hyperspace

Let the exploration domain combine continuous scenario parameters, categorical choices, relevant initial state, environment/configuration, and temporal or sequence features. Call this mixed domain Omega. The running legacy system induces an observation map F over Omega. Canonicalization and migration relevance define an equivalence relation over points that behave the same for the purpose of the migration.

$\Omega = X \times E \times S_0 \times T$
 $F : \Omega \rightarrow Y$
 $z \sim z' \text{ iff } \text{Canon}(F(z)) == \text{Canon}(F(z'))$
 and required effect/invariant classes agree

BehaviorPlane P_k = connected component of $[z]_{\sim}$

The term behavior plane is intentionally geometric but should not be read as a claim that every region is globally affine. In the general case P_k may be a curved manifold, stratified region, or mixed discrete/continuous cell. A plane is a useful local chart: within a sufficiently stable neighborhood, one behavioral law, effect signature, and invariant family define the node. The graph is therefore the quotient/adjoining structure of the hyperspace rather than a collection of sampled points.

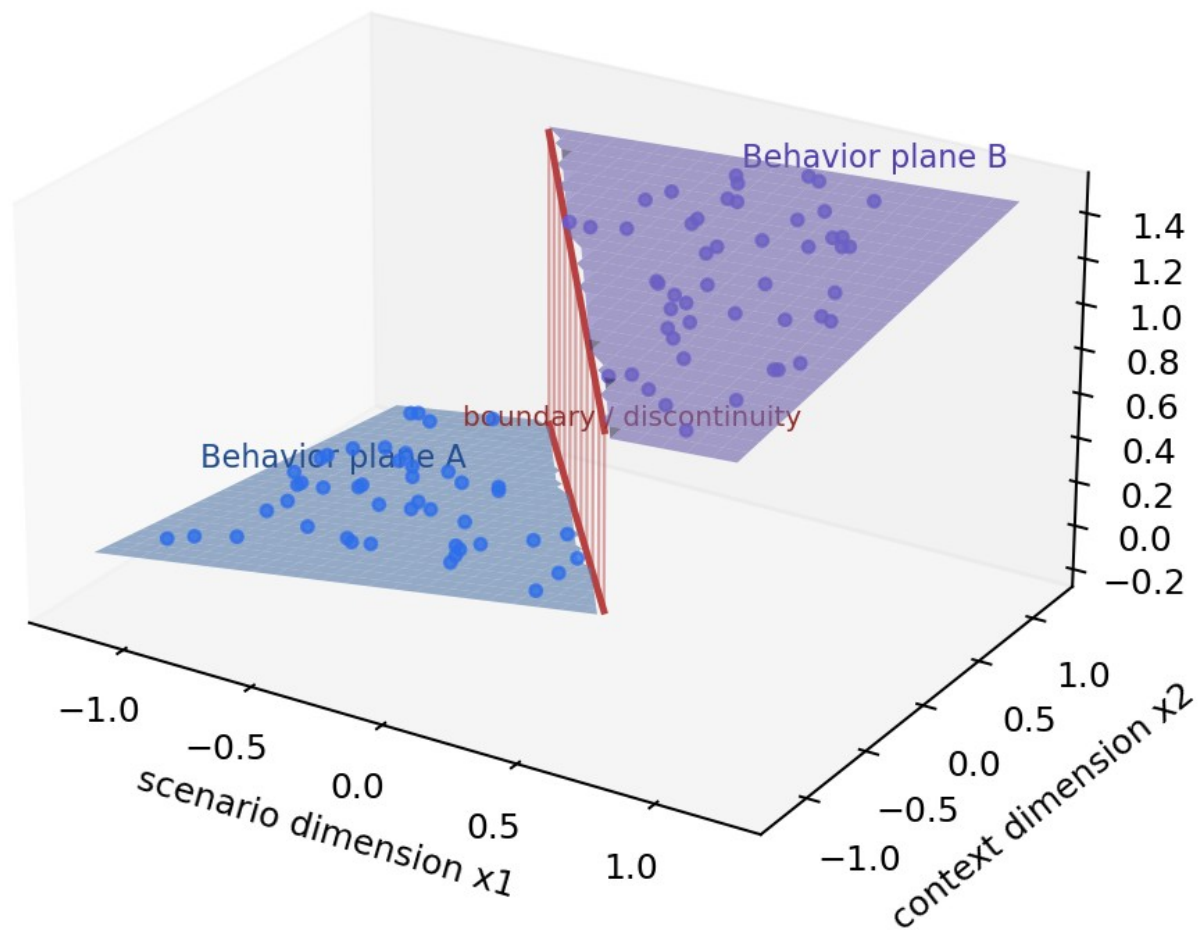


Figure 3. A low-dimensional slice of the behavioral hyperspace. Monte Carlo observations lie on locally stable behavior planes. A discontinuity surface separates planes; crossing it creates graph adjacency and eventually a guard/edge in MGIR.

Edge(P_i, P_j) exists when an admissible execution or perturbation can cross
 $B_{ij} = \text{closure}(P_i) \text{ intersect } \text{closure}(P_j)$
 and the canonical observation changes from behavior class i to j .

Geometric interpretation

A node says "inside this region, behavior follows this semantic law." An edge says "these behavior regions are reachable neighbors or state-transition successors." A boundary model says where and under which context the law changes.

4. Witnessed State and Observation Semantics

The runtime state of a real application is too large to copy into a graph. MGIR therefore stores a semantic projection: the coordinates needed to locate an execution in the behavioral hyperspace, plus the parts of state that are relevant to its observed behavior and migration obligation. A witnessed execution is a sample on a behavior plane; the node represents the region inferred from many such witnesses.

WitnessedState $S = (\text{DomainProjection}, \text{EnvironmentProjection}, \text{ObservationClass}, \text{EvidenceSet})$

4.1 Canonical observation

For an E2E scenario q executed in environment e , define an observation function:

$O(\text{system}, q, e) = (\text{output}, \text{stateDelta}, \text{effectSignature}, \text{traceFingerprint}, \text{diagnostics})$

Canonicalization removes noise that should not define compatibility: timestamps, generated IDs, ordering that is semantically irrelevant, volatile headers, randomized salts, and other unstable fields. The canonical observation is what the graph compares and what differential conformance later validates.

4.2 State equivalence is purpose-relative

Two concrete executions can be treated as the same graph state when they are indistinguishable under the migration-relevant observation model. This avoids graph explosion while preserving behavior that matters. The equivalence relation is therefore explicit and versioned: changing the observation model may split an existing state class.

5. Evidence, Provenance, and Confidence

A graph generated by agents is useful only if its claims are auditable. Every state, edge, guard hypothesis, effect mapping, and invariant should point to the evidence that caused it to exist.

Evidence kind	Examples	Role
E2E witness	scenario inputs, outputs, persisted-state diff	Primary semantic evidence.
Monte Carlo sample	sample distribution, seed, coordinates, repeated observations	Maps region occupancy, rare classes, and stochastic behavior.
Runtime trace	spans, branch IDs, call stack, structured logs	Localizes behavior and distinguishes paths.
Effect observation	DB write set, file writes, HTTP calls, events	Captures externally relevant side effects.
Static hint	CFG edge, symbol reference, data-flow path	Guides exploration; does not prove runtime behavior.
Differential run	legacy vs target canonical observation	Validates migration conformance.
Human assertion	domain rule, accepted compatibility exception	Adds intent with explicit ownership.
Formal result	proof/check over lowered model	Strengthens properties of a graph abstraction.

5.1 Confidence levels

Level	Meaning	Allowed use
Hypothesized	Suggested by model/static clue; not executed.	Exploration only.
Observed	Witnessed in at least one controlled scenario.	Graph navigation; not yet a stable compile input.
Repeated	Reproduced across scenario variants or reruns.	Candidate normalization.
Bounded	Guard boundary or state partition has supporting counterexamples on both sides.	Stable branch semantics.
Conformant	Target implementation matches the graph witnesses and required invariants.	Eligible for migration gate.

Level	Meaning	Allowed use
Proven (model)	Property proven over a formalized graph subset.	Strongest property of the abstract model, not automatically of arbitrary legacy code.

Open-world rule

Absence of an observed branch is not evidence that the branch does not exist. MGIR should track unexplored regions and confidence explicitly.

6. Grey-Box E2E Exploration

Grey-box exploration sits between static program analysis and pure differential testing. It treats externally visible behavior as authoritative while using internal signals to choose better experiments. This is important for migration because the objective is not maximum code coverage by itself; it is maximum discovery of behaviorally distinct states, transitions, effects, and invariants.

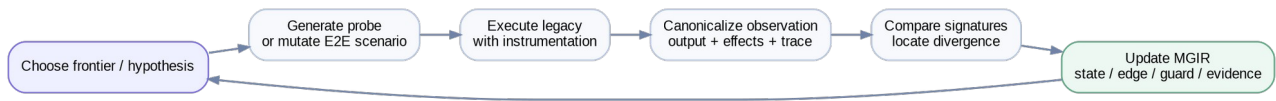


Figure 5. Knowledge-acquisition loop. Each experiment should reduce graph uncertainty or validate an existing claim.

6.1 Instrumentation spectrum

- Low intrusion: request/response capture, database snapshots, file diffs, queues, outbound HTTP, structured logs.
- Moderate intrusion: distributed traces, method/branch coverage, correlation IDs, selected variable probes, domain event capture.
- High intrusion when justified: targeted hooks around suspected predicates, fault injection, sandboxed dependency substitution, or deterministic clocks.
- Static hints: call graph, CFG, symbol references, data flow, schema relationships, dependency injection resolution, and configuration search.

Instrumentation should be mounted as DSH capabilities where practical, allowing a scenario to acquire a richer observation without hard-wiring the migration system to one application stack.

7. Discovering Branch Boundaries by Behavioral Change

One recurring idea is to search for discontinuities or sharp changes in observable behavior. The legacy application is treated as a function over scenario parameters and environment. We are not trying to differentiate arbitrary code analytically; we are using differences between executions to detect candidate decision boundaries.

7.1 Behavioral difference signal

```

sigma(x) = canonicalSignature(O(Legacy, scenario(x), env))
Delta(x_i, x_j) = distance(sigma(x_i), sigma(x_j))
  
```

The distance function can combine categorical and structured differences: response class, normalized payload, effect set, trace fingerprint, terminal state, and selected invariants. A large or qualitatively different Delta marks a candidate boundary.

7.2 Scan first, refine second

A practical search strategy is hierarchical. A broad scan samples the domain and stores intervals where the signature changes. Refinement then concentrates probes inside those intervals. For ordered numeric inputs, bisection or change-point search is generally safer than Newton-Raphson because the target is often a discontinuity or discrete branch. Newton-like methods become useful only when a continuous surrogate or differentiable guard function has been identified.

coarse scan -> changed interval [a,b] -> adaptive refinement -> candidate guard -> counterexample probes

7.3 Multidimensional inputs

For $f(x_1, x_2, \dots, x_n)$, exhaustive grids are usually impossible. The graph supplies structure for adaptive sampling. The explorer can mutate one dimension around a witnessed state, use pairwise/combinatorial designs, prioritize variables correlated with trace divergence, or ask an agent to generate hypotheses from static predicates. The goal is not to enumerate R^n ; it is to identify semantically meaningful partitions and preserve the evidence that defines them.

7.4 Monte Carlo observations over the hyperspace

Monte Carlo sampling supplies the broad exploration layer. Draw scenario/context points from an explicit proposal distribution $q(z)$, execute them through DSH, and persist both the sample coordinate and canonical observation. The distribution may mirror production traffic, deliberately flatten rare dimensions, stratify known categories, or be biased toward existing frontiers. Because q is recorded, observations can be reweighted later rather than pretending that a convenience sample represents real-world frequency.

```
z_i ~ q(z), i = 1..M
sigma_i = Canon(O(Legacy, scenario(z_i), env(z_i)))
record (z_i, sigma_i, trace_i, effects_i, q(z_i), seed_i)
```

There are two useful Monte Carlo axes. Across the hyperspace, random samples map behavior regions and discover rare classes. At a fixed coordinate z , repeated executions sample runtime nondeterminism, allowing the node to represent a distribution $P(\text{observation} \mid z)$ rather than incorrectly splitting random outcomes into deterministic branches. Classical Monte Carlo establishes the statistical basis for learning from repeated random evaluations [11].

7.5 Derivative-like fields as discontinuity detectors

A true discontinuity does not have an ordinary derivative at the jump. The migration explorer should therefore use finite-difference or perturbation scores as detectors, not claim an analytic gradient of the program. Let ϕ embed a canonical observation into a numeric feature vector, or let d compare two canonical signatures directly. For a local direction v :

$$D_h(z; v) = d(\sigma(z + h v), \sigma(z - h v)) / (2h)$$

If the two probes remain on one behavior plane: D_h is small or follows the plane local model.

If they straddle a jump of size J : D_h is approximately $J/(2h)$, so the score spikes as h shrinks.

Coordinate-wise finite differences cost $O(n)$ or $O(2n)$ executions per point in n continuous dimensions. In high-dimensional systems, random-direction or simultaneous perturbation estimates can probe many coordinates with two executions per direction, borrowing the efficiency intuition of simultaneous perturbation stochastic approximation [12]. Categorical variables use a discrete derivative instead: mutate one category or flag and measure semantic distance.

$$\delta_j(z) = d(\sigma(z), \sigma(\text{mutate_dimension}_j(z)))$$

random-direction score: choose $v \sim \text{DirectionDistribution}$; evaluate $z \pm h v$

The derivative field is therefore an experimental field over the behavioral hyperspace. Its peaks identify candidate boundary neighborhoods; its dominant directions suggest a local boundary normal and which variables most strongly control the branch. Variance-based global sensitivity estimates can complement this by identifying dimensions and interactions that explain output variation under Monte Carlo sampling [13].

7.6 Adaptive Monte Carlo boundary acquisition



Figure 4. Monte Carlo plus derivative-guided boundary acquisition. Broad random observations discover behavior planes; nearby cross-class pairs and finite-difference spikes seed importance sampling and local boundary refinement.

The useful composition is global randomness followed by local determinism. Monte Carlo prevents the explorer from committing too early to an agent hypothesis. Once samples reveal nearby points with different behavior classes, the interpreter treats the segment or neighborhood between them as a boundary seed. It then allocates more probes there, uses bisection/change-point refinement on continuous dimensions, and constructs counterexamples on both sides.

1. sample broadly from $q(z)$
2. cluster samples into behavior-plane candidates
3. find near-neighbor pairs (z_a, z_b) with $\text{class}_a \neq \text{class}_b$
4. evaluate D_h around those neighborhoods and random directions
5. importance-sample where D_h or class entropy is high
6. refine boundary B_{ab} and fit a local separator / guard hypothesis
7. validate both sides + nearby tangential perturbations
8. create/update MGIR edge and boundary evidence

A locally flat boundary can be represented by a hyperplane $n^T(z - b) = 0$ and compiled as a guard. Curved or mixed boundaries require richer region predicates, decision trees, symbolic expressions, or piecewise surfaces. The IR should retain the samples and confidence behind that fitted boundary so later counterexamples can split or replace it.

7.7 The graph as the adjacency quotient of sampled behavior

This closes the geometric loop. Monte Carlo witnesses are points in Ω ; the reducer groups connected, semantically equivalent points into behavior planes P_k ; derivative/change analysis estimates the separating surfaces B_{ij} ; and MGIR stores only the quotient structure needed for migration - region semantics, adjacency, guards, effects, evidence, and unresolved boundary segments. Sampling remains attached as provenance but does not become one graph node per execution.

8. State Inference, Equivalence Classes, and Invariants

Repeated observations can collapse large input spaces into behavior classes. If many scenarios produce the same canonical observation and trace/effect family, they are candidates for the same state region. Divergent observations split the region. This resembles active model learning and dynamic specification mining, but the graph remains migration-oriented and intentionally hybrid: it can combine inferred behavior with explicit domain annotations and source-code localization [4][5].

8.1 Likely invariants

The explorer can mine candidate invariants from traces and state snapshots: ranges, equalities, orderings, monotonic relationships, membership constraints, pre/post conditions, and stable effect relations. Work such as Daikon demonstrates the usefulness of dynamically inferred likely invariants for program understanding and evolution [3]. MGIR should store them as hypotheses until enough evidence or static/formal checking raises their confidence.

8.2 Coverage becomes semantic

Metric	Question
State evidence coverage	Which semantic state classes have direct witnesses?
Transition coverage	Which outgoing transitions have been executed?
Guard boundedness	Do we have counterexamples on both sides of a proposed boundary?
Effect coverage	Are externally relevant effects mapped for each transition?
Invariant support	How many independent witnesses support each likely invariant?
Monte Carlo support	How much weighted sample mass supports each behavior plane and which dimensions remain weakly sampled?
Boundary sharpness	Where do finite-difference/change scores spike and how stable is the fitted surface across h and reruns?
Frontier count	Which graph regions remain explicitly unresolved?

Metric	Question
Differential conformance	Which graph edges have legacy/target equivalence evidence?

8.3 Building nodes from planes rather than points

The reducer should avoid turning every observation into a state node. It first clusters observations by canonical behavior and transition/effect family, then checks local connectedness in the sampled hyperspace. A single node represents the supported behavior region and can carry a local model such as an affine chart, decision region, symbolic guard conjunction, or probabilistic response law. Two disconnected regions with identical immediate output may still remain separate nodes when their reachable successors or effects differ.

9. DeepSeek Harness as the Execution Substrate

DSH already exposes the properties this architecture needs: plugin-composed capabilities, guarded tool execution, sessions, subagents, goals, workflows, sandboxes, and an append-only session event log from which resume, fork, replay, and transcripts are derived [1][2]. MGIR should use those capabilities rather than duplicating them.

9.1 Proposed DSH plugin roles

Plugin role	Responsibility
MGIR store	Persist graph, evidence references, versions, confidence, indexes.
Scenario source	Load curated E2E scenarios and generate parameterized variants.
Scenario runner	Execute legacy/target systems in controlled environments.
Trace/effect adapters	Collect code coverage, spans, DB/file/HTTP/event effects.
Static-hint adapter	Expose CFG/data-flow/symbol clues to the explorer.
Monte Carlo sampler	Draw global, stratified, or importance-weighted hyperspace probes.
Boundary estimator	Compute local finite-difference/change scores and refine candidate discontinuities.
Frontier planner	Rank unknown graph regions and propose next probes.
Evidence reducer	Canonicalize observations and merge/split graph states.
Conformance validator	Run legacy-vs-target differential scenarios.
Compiler backend	Lower stable MGIR subsets to a target artifact.
Reviewer agents	Challenge inferred guards, missing branches, and evidence quality.

9.2 DSH session events as evidence provenance

DSH treats model-visible context and durable session facts as an append-only event stream. MGIR can reference exact session and event boundaries for each discovery run. A graph assertion can therefore point back to the scenario input, model hypothesis, tool calls, trace extraction, result, and validation that produced it. This creates a chain from high-level migration claim to replayable execution evidence.

9.3 Forking as parallel branch exploration

A graph frontier naturally maps to session forks or subagent tasks. Multiple agents can explore different unresolved branches from the same known state without each carrying the entire migration in prompt context. Their results merge at the graph/evidence layer, where contradictory observations become first-class conflicts rather than hidden prompt disagreements.

10. The Graph as an Intermediate Representation

The compiler framing changes the design. The legacy system is not translated directly into a target language. Instead, behavior is progressively lifted into a backend-neutral representation. The graph is interpreted while incomplete and compiled only after a region becomes sufficiently constrained.



Figure 6. Staged IR pipeline. Exploration tolerates uncertainty; compilation operates on normalized, validated subsets.

10.1 Three IR stages

Stage	Properties	Primary consumer
Exploratory IR (EIR)	Partial, contradictory facts allowed, confidence and provenance mandatory, explicit unknown frontiers.	Interpreter / exploration agents.
Normalized Migration IR (NIR)	Typed states, canonical guards, effect vocabulary, resolved state equivalence, explicit invariants.	Validators / planners / compiler passes.
Backend IR (BIR)	Target-specific types and operations after capability checking.	Emitter for code, tests, formal model, workflow, docs.

10.2 Why this is stronger than storing test results

A test result is a point observation. The IR expresses relationships: which observations are equivalent, which behavior plane they support, which boundary or state transition they witness, which guard partitions the inputs, which effects belong to that branch, what evidence supports the claim, and what remains unknown. Those relationships are exactly what a compiler or migration planner needs.

10.3 Geometric information survives only as long as it is useful

EIR may contain sample clouds, local derivative estimates, fitted separators, and uncertainty fields because the interpreter needs them to decide what to observe next. NIR should lower stable geometry into explicit region predicates, guards, probabilistic transition models, and proof/conformance obligations. A target backend does not need to understand Monte Carlo itself unless it is compiling an exploration workflow; it needs the normalized semantics that the sampling justified.

11. Interpretation During Exploration

The EIR must be executable even when incomplete. The interpreter is not a runtime for the target program; it is a runtime for knowledge acquisition and migration planning.

11.1 Interpreter responsibilities

- Resolve the current graph state from scenario setup and known evidence.
- Select a frontier, low-confidence edge, disputed invariant, high-derivative boundary neighborhood, or high-value migration dependency.
- Instantiate an experiment from the graph: Monte Carlo draw, importance-sampled boundary probe, input mutation, state setup, instrumentation set, and observation contract.
- Invoke DSH tools/subagents to execute and inspect the run.
- Canonicalize the result, compare it with existing signatures, and update graph structure or confidence.
- Produce contradictions instead of overwriting evidence when new behavior disagrees with an existing hypothesis.

11.2 Active information gain

Probe selection can be framed as expected information gain rather than raw coverage maximization:

$$\begin{aligned}
 \text{score}(q) = & \text{ExpectedUncertaintyReduction}(q) / \text{Cost}(q) \\
 & + \lambda * \text{MigrationRelevance}(q) \\
 & - \mu * \text{ExecutionRisk}(q)
 \end{aligned}$$

An agent can estimate this score qualitatively. A later implementation can formalize uncertainty from graph entropy, confidence distributions, frontier size, or branch-bounding progress. This turns exploration into a directed learning problem.

12. Compilation to Multiple Backends

MGIR should be backend-neutral, but "compile to any backend" should be understood in the compiler sense: any target for which a semantic lowering is implemented. A backend declares which MGIR concepts it can represent, and compilation fails explicitly when required semantics are unsupported.

Backend	Possible output	Use
Conformance tests	Playwright/API tests/custom E2E fixtures/property tests	Lock observed semantics against regressions.
Target implementation	C#, Go, TypeScript, Rust or domain-specific modules	Generate pure logic, state machines, region dispatchers/guards, adapters, schemas, or scaffolding where semantics are explicit enough.
DSH workflow	Goals, frontier tasks, validation plans, subagent work queues	Execute migration steps directly from graph intent.
Formal model	Lean inductive transitions, TLA+ actions/invariants, model-checker input	Reason about abstract preservation, reachability, forbidden states.
Documentation	State diagrams, decision tables, compatibility contracts, ADR material	Human review and domain validation.
Data migration	Mapping tables, transforms, validation queries, backfill plans	Move persisted state with traceable semantic obligations.

12.1 Backend capability negotiation

```
BackendCapabilities {
  supportsGuards
  supportsStatefulTransitions
  supportsEffects
  supportsNondeterminism
  supportsUncertainty // usually false for code emitters
  supportsProofObligations
}
```

The exploratory graph may contain uncertainty and unresolved effects. A target-code backend should reject such regions or require an explicit fallback/adapter. A documentation backend can render them. A formal backend may turn them into assumptions. This prevents silent semantic loss during lowering.

13. Migration by Pure Semantic Regions

Once a behavior plane or branch region is behaviorally stable, the migration can extract a pure kernel. Environmental dependencies become explicit inputs, and external effects become effect intents or adapters. This is not required for MGIR, but it makes backend compilation and equivalence checking much easier.

```
PureKernel : (DomainState, Input, DeclaredContext)
  -> (DomainState', Output, EffectIntents)
```

The graph tells us when this extraction is justified: a region has stable guards, repeated observations, mapped effects, and no unresolved internal branch that changes semantics. The migrated implementation can then be tested against all witnesses assigned to that region.

13.1 Why this helps legacy decomposition

- Behavioral regions can be migrated independently even when the legacy code interleaves them in one function or call chain.

- Effect isolation makes differential comparison more deterministic and target-language generation more portable.
- The graph provides a decision structure for routing inputs to the correct migrated region during a strangler-style transition.
- Unmigrated branches remain executable through the legacy backend while migrated branches route to the new implementation.

14. Worked Example: Discovering a Hidden Pricing Branch

Consider a legacy pricing service being migrated to a new backend. Documentation says price depends on quantity and customer tier. A curated E2E scenario exercises a standard customer with quantity 10 and returns 90.00.

14.1 Exploration

Probe	Observation	Trace/effect clue	Graph update
qty=10, standard	90.00	Path A; no DB write	Witness S0 -> PriceA.
qty=11, standard	99.00	Path A	Same region.
qty=20, standard	160.00	Different branch ID; promo lookup	Split region; candidate threshold.
qty=19, standard	171.00	Path A	Boundary interval [19,20].
qty=20, standard, promo disabled	180.00	Same branch, different configuration	Environment predicate added.
qty=20, gold	150.00	Additional tier branch	New outgoing edge and frontier.

The first surprising result does not immediately become a rule. It becomes a hypothesis supported by one witness. Refinement probes bound the quantity threshold. Environment mutation reveals that the branch also depends on promotion configuration. Static analysis points to the relevant predicate, but the E2E witnesses define the compatibility semantics.

14.2 Normalized graph fragment

```

State: PricingInput

edge A:
  guard: quantity < 20 OR promoEnabled == false
  action: basePricing
  effects: none

edge B:
  guard: quantity >= 20 AND promoEnabled == true
  action: promotionPricing
  effects: promotionLookup

edge C:
  guard: customerTier == gold
  action: tierAdjustment
  status: frontier (interaction with edge B not fully bounded)

```

A code-generation backend can compile A and B only after their precedence and interaction are normalized. The documentation backend can render C as unresolved. The interpreter prioritizes probes around the B/C interaction because it is both uncertain and migration-relevant.

14.3 Monte Carlo and hyperspace interpretation

Represent the pricing scenario by $z = (\text{quantity}, \text{promoEnabled}, \text{customerTier}, \text{relevant account state}, \text{environment})$. A Monte Carlo batch samples valid combinations rather than scanning quantity alone. Most observations collapse onto a small number of behavior planes. For $\text{promoEnabled}=\text{false}$, the directional change score along quantity

remains on the base-pricing plane across quantity 20. For `promoEnabled=true`, symmetric probes around the threshold straddle two canonical signatures and `D_h` spikes. Toggling the promo flag provides a discrete derivative that is near zero below the threshold and large at or above it.

```
z = (quantity, promoEnabled, tier, state, env)

D_qty(z; h) = d(sigma(qty+h), sigma(qty-h)) / (2h)
delta_promo(z) = d(sigma(promo=false), sigma(promo=true))

observed pattern -> boundary hypothesis: quantity >= 20 AND promoEnabled == true
```

The graph node for base pricing is therefore a supported region/plane, not the individual `qty=10` or `qty=19` samples. Promotion pricing is another plane. Their boundary is the intersection of a quantity surface with a configuration predicate. Gold-tier interactions can create additional planes or split an existing one when the Monte Carlo observations and boundary probes show different successor behavior.

15. Differential Conformance as the Migration Gate

The target implementation should not be declared correct because it passes newly written unit tests. It should be evaluated against the evidence-backed semantic graph. For each critical graph edge, execute equivalent scenarios against legacy and target systems and compare canonical observations.

```
Conform(edge) := canon(O(Legacy, q)) ~ canon(O(Target, q))
AND requiredInvariants(Target, q)
AND effectPolicy(edge)
```

Differential testing is an established technique for exposing discrepancies by comparing independent implementations [6]. Here it becomes graph-indexed: failures are associated with a specific semantic edge, guard region, and evidence set, which makes regression diagnosis and migration progress measurable.

15.1 Compatibility does not always mean identity

The comparator must support approved semantic differences. A migration may intentionally improve latency, remove an obsolete side effect, normalize an error code, or fix a historical bug. Such changes become explicit graph annotations or human-approved invariants rather than ad hoc comparator exceptions.

16. Formal Verification as a Backend, Not the Source of Truth

MGIR can be lowered into Lean or another formal language to reason about the abstract transition system. This is valuable because the graph already contains states, transitions, guards, invariants, and forbidden conditions. But the trust boundary must remain explicit: proving a theorem over the graph does not automatically prove that arbitrary legacy code implements that graph.

```
inductive State
| s0 | s1 | s2 | ...

inductive Step : State -> State -> Prop
| branchA : GuardA x -> Step s0 s1
| branchB : GuardB x -> Step s0 s2

def Invariant : State -> Prop := ...

theorem preservation : Step s t -> Invariant s -> Invariant t := ...
```

The useful loop is bidirectional: dynamic discovery proposes a model; formal tooling proves properties of that model; generated conformance tests check that both legacy and target implementations remain observationally aligned with it. Research combining dynamically inferred invariants with static verification illustrates the same complementary principle [7].

17. Comparison with Adjacent Approaches

Approach	Strength	Limitation for migration	MGIR contribution
Static analysis / CFG	Broad source coverage, precise syntax relations.	Dynamic dispatch, environment, effects, and semantic relevance can explode.	Uses static results as hints and localization metadata.
Black-box E2E	Architecture-neutral semantic oracle.	Expensive exploration; poor localization.	Adds traces/static hints and persistent state/branch model.
Record/replay	Fast regression of known traffic.	Replays points, not decision boundaries or unknown regions.	Generalizes traces into state classes, guards, and frontiers.
Unit-test preservation	Fine-grained and deterministic.	Often preserves obsolete architecture rather than external semantics.	Compiles new tests from backend-neutral behavior model.
LLM-only code exploration	Flexible reasoning over heterogeneous code.	Coverage, memory, and evidence discipline are unreliable.	Externalizes discoveries into a durable IR with provenance.
Active automata/model learning	Systematic inference of finite-state behavior.	May require strict alphabets/equivalence queries and black-box assumptions.	Adopts active learning ideas but allows grey-box hints, partial graphs, rich effects, and migration annotations.
Specification mining	Can infer likely invariants and protocols.	Mined rules may be architecture- or trace-dependent.	Stores mined rules with confidence, counterexamples, and compilation semantics.

18. Implementation Architecture

18.1 Suggested MGIR storage model

```

GraphStore
nodes(id, kind, semanticKey, projectionSchema, confidence)
edges(id, src, dst, guardExpr, actionRef, effectSet, confidence)
invariants(id, scope, predicate, confidence)
frontiers(id, scope, reason, priority)
evidence(id, kind, sessionId, eventRange, scenarioId, artifactHash)
evidence_links(subjectType, subjectId, evidenceId, supportOrRefute)
code_links(subjectId, repoCommit, file, symbol, range, relation)
observations(id, scenarioId, coordinateVector, canonicalSignature, proposalDensity, seed, artifactRefs)
behavior_regions(id, nodeId, coordinateSchema, localModel, supportStats)
boundaries(id, nodeA, nodeB, model, derivativeStats, confidence)
samples(id, observationId, regionId, weight, neighborIndexRef)

```

A property graph database is natural for traversal, but relational storage with explicit edge tables can be sufficient and easier to version. Large traces, payloads, screenshots, or database snapshots should live in content-addressed artifact storage and be referenced by hash rather than embedded in the graph.

18.2 Versioning

Graph evolution should be append-oriented. A new observation can supersede a hypothesis without deleting the history that justified it. Graph revisions should bind to legacy commit/version, environment fingerprint, scenario version, and observation canonicalizer version. That allows the same semantic region to be compared across releases or migration phases.

18.3 Contradictions

Contradictory witnesses are valuable. They may expose hidden environment state, nondeterminism, data dependence, or an incorrect equivalence class. The reducer should create a conflict/frontier and ask the interpreter for discriminating experiments rather than averaging the conflict away.

18.4 Sampling and boundary indexes

Monte Carlo exploration needs indexes that the graph itself does not provide: nearest-neighbor lookup in normalized coordinate space, sample weights, validity constraints, local class entropy, and cached perturbation pairs. These can live beside the graph as an analysis index. The durable MGIR node/edge remains backend-neutral while EIR references the sample cloud and boundary-estimation artifacts that justify it.

19. Practical Build Roadmap

Phase	Deliverable	Exit criterion
0. DSH integration	MGIR store plugin + scenario runner + evidence links to DSH sessions.	One manual E2E run creates a replayable graph witness.
1. Observation model	Canonical output/effect/trace signature and diff engine.	Repeated identical scenarios converge to same signature.
2. Statistical exploration	Monte Carlo/stratified sampler + repeated-run nondeterminism estimates.	System discovers multiple behavior classes without a hand-authored grid.
3. Boundary acquisition	Finite-difference/random-direction estimator + importance sampler + segment refinement.	System localizes and bounds at least one hidden discontinuity surface.
4. State/plane reducer	Merge/split connected behavior regions; fit local region models; confidence and contradiction handling.	Graph remains compact under hundreds of runs and samples map to planes instead of point nodes.
5. Static hints	CFG/symbol/data-flow adapters correlated to witnesses.	Explorer localizes divergent behavior to code regions.
6. Conformance backend	Legacy-vs-target graph-indexed differential runner.	Migration status measurable per edge/state.
7. Compiler API	EIR -> NIR passes + backend capability contract.	At least two independent backends share same normalized IR.
8. First emitters	Conformance-test backend + documentation/state-diagram backend.	Stable graph region emits useful artifacts without model reinterpretation.
9. Formal/backend experiments	Lean/TLA+ model + target-language pure-region generator.	Semantics that cannot lower fail explicitly instead of being dropped.

Best first experiment

Choose one legacy E2E flow with a poorly documented numeric/configuration branch. Start with a Monte Carlo or stratified batch, show that observations collapse into distinct behavior planes, use finite-difference/change scores to localize the discontinuity, persist the boundary as MGIR evidence, then regenerate a conformance test from the graph.

20. Research Questions

- How should semantic distance be defined so that a probe prioritizes meaningful behavior changes over noisy implementation differences?
- How should the Monte Carlo proposal distribution balance production-likelihood, rare-state discovery, and deliberately adversarial coverage?
- How should finite-difference step size h adapt to noise, discrete quantization, and state-reset cost without smearing narrow boundaries?
- Which random-direction or simultaneous-perturbation estimators best recover boundary normals in mixed continuous/categorical spaces with expensive E2E evaluations?

- When should a node be modeled as a local affine plane, a curved manifold, a symbolic region, or a probabilistic behavior law?
- How can the reducer infer minimal state partitions without collapsing rare but important compatibility branches?
- What confidence model best combines static hints, dynamic witnesses, repeated runs, differential conformance, and human domain assertions?
- How should active exploration balance information gain, scenario execution cost, environment reset cost, and production-like risk?
- Can graph frontiers be scheduled across subagents so that parallel exploration maximizes non-overlapping information gain?
- Which MGIR operations are sufficiently universal to support code generation across languages without turning the IR into a low-level AST?
- How should nondeterministic or probabilistic behavior be represented and validated?
- Can formal-model counterexamples be translated back into concrete E2E probes, closing a formal-to-runtime loop?
- How should the compiler expose unsupported semantics so a human or agent can decide whether to refine the graph, add an adapter, or approve a deliberate incompatibility?

21. Limits and Failure Modes

This architecture reduces uncertainty; it does not eliminate the fundamental incompleteness of dynamic observation. A graph can only represent behavior that was observed, inferred, declared, or proven at its abstraction level.

- Rare state/data combinations may remain undiscovered. Frontier accounting helps make that ignorance visible but cannot prove absence.
- Monte Carlo coverage depends on the proposal distribution. A rare behavior plane can have near-zero probability under ordinary traffic and remain invisible unless sampling is stratified or frontier-biased.
- Finite-difference spikes are detectors, not derivatives at the discontinuity itself. Their magnitude depends on step size, canonicalization, observation noise, and whether the two probes actually straddle the boundary.
- High-dimensional coordinates are often constrained and correlated; naive perturbations can generate invalid scenarios. The sampler needs domain-valid proposal kernels and state-reset semantics.
- Behavior planes are local abstractions. Curved, disconnected, history-dependent, or chaotic regions may require manifold/piecewise/probabilistic models rather than a single fitted hyperplane.
- Grey-box instrumentation can perturb timing or behavior. The evidence model should record instrumentation configuration and support lower-intrusion confirmation runs.
- Legacy nondeterminism can produce false branch splits. Canonicalization and repeated witnesses are required before normalization.
- Overly detailed observation signatures can explode the graph; overly coarse signatures can merge incompatible states.
- Code generation is constrained by IR expressiveness. The compiler must reject unsupported semantics rather than fabricate them.
- Formal verification applies to the lowered model and any verified connection to implementations; it is not magic proof of an unmodeled legacy system.
- Agent-generated hypotheses are optimization hints until they are backed by execution evidence or explicit domain approval.

22. Conclusion

The central proposal is to stop treating legacy migration as a sequence of code-edit tasks and treat it as progressive compilation of acquired knowledge. The legacy system is the behavioral source. Grey-box end-to-end scenarios are the measurement instrument. Monte Carlo sampling maps the behavioral hyperspace; finite-difference and random-direction change fields expose its discontinuity surfaces. MGIR is the semantic intermediate representation of the

resulting behavior planes and their adjacency. DSH is the runtime that interprets the partial graph, performs experiments, records provenance, and coordinates agents. Compiler backends turn stable knowledge into migration artifacts.

This framing gives the migration process a durable object that can outlive any model turn, agent session, target language, or implementation strategy. A behavior plane or branch boundary discovered once becomes reusable knowledge. A failed hypothesis becomes a recorded counterexample. An unexplored region remains visibly unresolved. A migrated region carries the witnesses that justify it. And the same graph can drive exploration today, conformance tomorrow, and formal or code-generation backends later.

End state

The desired artifact is not "the agent understood the legacy code." It is a backend-neutral, evidence-backed graph that another agent, compiler, reviewer, or verification tool can execute and interrogate without rediscovering the system.

References

- [1] DeepSeek AI. DeepSeek Harness - official project and developer preview. 2026. <https://github.com/deepseek-ai/deepseek-harness> ; <https://deepseek.com/harness/en/>
- [2] DeepSeek AI. DeepSeek Harness Architecture. 2026. DSH documentation: Cordis plugin composition, append-only SessionEvent log, tools, agents, sessions, and capability seams. <https://github.com/deepseek-ai/deepseek-harness/blob/master/docs/architecture.md>
- [3] Ernst, M. D.; Perkins, J. H.; Guo, P. J.; McCamant, S.; Pacheco, C.; Tschantz, M. S.; Xiao, C. "The Daikon system for dynamic detection of likely invariants." *Science of Computer Programming* 69(1-3), 2007, pp. 35-45.
- [4] Aichernig, B. K.; Tappler, M. "Efficient Active Automata Learning via Mutation Testing." *Journal of Automated Reasoning* 63, 2019, pp. 1103-1134.
- [5] Ernst, M. D.; Cockrell, J.; Griswold, W. G.; Notkin, D. "Dynamically discovering likely program invariants to support program evolution." *IEEE Transactions on Software Engineering* 27(2), 2001, pp. 99-123.
- [6] McKeeman, W. M. "Differential Testing for Software." *Digital Technical Journal* 10(1), 1998, pp. 100-107.
- [7] Nimmer, J. W.; Ernst, M. D. "Static verification of dynamically detected program invariants: Integrating Daikon and ESC/Java." *Electronic Notes in Theoretical Computer Science* 55(2), 2001, pp. 255-276.
- [8] Shi, Z.; Zhang, Y.; Cui, et al. "A Programming Paradigm for Spatiotemporal Composability." Cordiverse preprint, draft 13 Aug 2026. <https://github.com/cordiverse/paper> . Relevant as the Cordis foundation beneath DSH; MGIR is a higher-level migration representation rather than an alternative plugin runtime.
- [9] Roslyn FlowAnalysis / ControlFlowGraph APIs. Microsoft .NET compiler analysis documentation. <https://learn.microsoft.com/dotnet/api/microsoft.codeanalysis.flowanalysis>
- [10] CodeQL control-flow and data-flow analysis documentation. <https://codeql.github.com/docs/>
- [11] Metropolis, N.; Ulam, S. "The Monte Carlo Method." *Journal of the American Statistical Association* 44(247), 1949, pp. 335-341. doi:10.1080/01621459.1949.10483310.
- [12] Spall, J. C. "Multivariate stochastic approximation using a simultaneous perturbation gradient approximation." *IEEE Transactions on Automatic Control* 37(3), 1992, pp. 332-341. doi:10.1109/9.119632.
- [13] Sobol, I. M. "Global sensitivity indices for nonlinear mathematical models and their Monte Carlo estimates." *Mathematics and Computers in Simulation* 55(1-3), 2001, pp. 271-280. doi:10.1016/S0378-4754(00)00270-6.

Terminology

Term	Meaning in this paper
DSH	DeepSeek Harness, used as the runtime/plugin/session substrate.
MGIR	Migration Graph Intermediate Representation, the backend-neutral semantic graph.
Behavior plane	A connected region/local manifold in scenario-context hyperspace where one migration-relevant behavioral law holds.
Behavioral hyperspace	Mixed-dimensional domain of inputs, initial state, environment/configuration, and temporal/sequence coordinates explored by scenarios.
Boundary field	Finite-difference/discrete-change scores used to detect neighborhoods where canonical behavior changes.
EIR	Exploratory IR: partial, uncertain, evidence-rich MGIR stage.
NIR	Normalized Migration IR: typed and sufficiently constrained for validation/lowering.
BIR	Backend IR: target-specific representation after lowering.
Grey-box E2E	End-to-end behavioral testing guided by internal traces/static/runtime signals.
Witness	A concrete execution that supports or refutes a graph claim.
Frontier	An explicitly unresolved graph region or hypothesis requiring more evidence.